

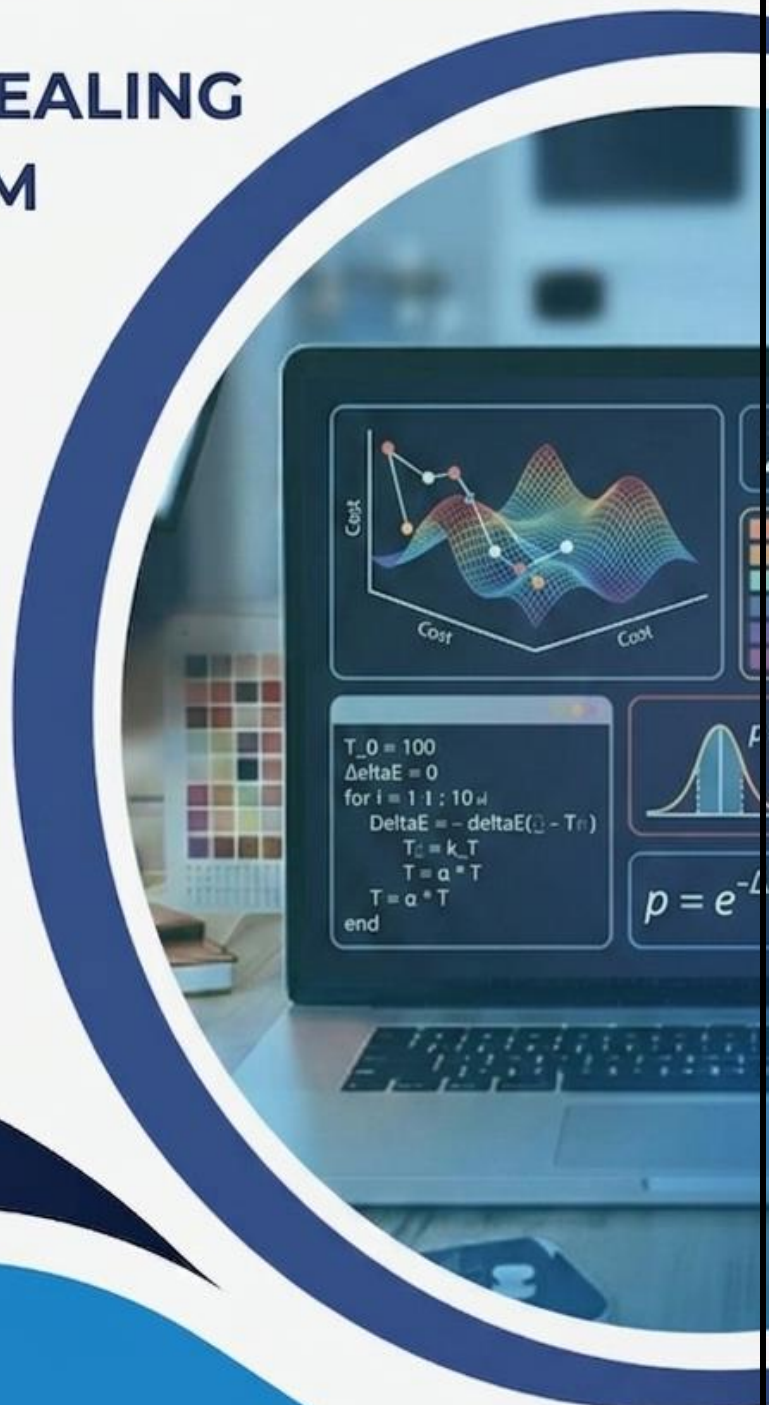
PROJECT REPORT

SIMULATED ANNEALING ALGORITHM

ANALYSIS OF ALGORITHM

INSTRUCTOR

USMAN SHARIF



PRESENTED BY:-

NAME :- AHMED ALI (65235)



Table of Contents

1	Introduction (CLO 1.1 – Understand).....	3
A.	Algorithm	3
B.	Problems Solved.....	3
C.	Algorithm Design Strategy	3
D.	Real-World Motivation.....	4
E.	Why Simulated Annealing?	4
2	Algorithm Explanation (CLO 1.1 – Understand).....	4
A.	Step By Step Working :-	4
B.	PsuedoCode Of Simulated Annealing	5
C.	FlowChart :-.....	6
		7
3	Implementation (CLO 2.1 – Implement)	8
A.	Programming Language:-	8
	Python (Traveling Salesman Problem):-.....	8
	JavaScript (N-Queen Problem):-	8
B.	Code Snippet :-.....	8
4	Empirical Analysis (CLO 2.1 – Implement)	10
A.	Inputs Used :-	10
B.	Environment:-	11
C.	Observation:-	11
	Growth Trend:-.....	11
	Performance:-	11
5	Complexity Analysis (CLO 3.1 – Analyze	12
A.	Time Complexity :_	12
	Best Case:.....	12
	Average Case:.....	12
	Worst Case Time Complexity:	12
B.	Space Complexity	13
	Auxiliary Space: $O(n)$	13
	Total Space: $O(n)$	13
6	Real-World Application (CLO 2.1 + 3.1)	13
A.	Applications:-	13

B.	Example: Delivery Route Optimization:-.....	14
C.	Trade-offs.....	14
	Time vs Memory Usage.....	14
	Conclusion:.....	14
	Accuracy vs Efficiency	14
	Conclusion:.....	15
7	Limitations & Challenges (CLO 3.1 – Analyze).....	15
A.	In case the algorithm does not produce a better solution	15
B.	2. Limitations and assumptions	15
C.	Comparison with Better Alternatives	15
8	Conclusion.....	16
	Key Findings:-	16
	Key Discoveries Gained from the Work Experience	16

Course	Analysis of Algorithms
Semester	Spring 2026
Student Name	Ahmed Ali
SAP ID	65235
Instructor	Mr. Muhammad Usman Sharif
Algorithm	Simulated Annealing (SA)
Problems	N-Queens Problem Travelling Salesman Problem (TSP)
Language	JavaScript (Vanilla + Node.js / Express.js)

CLO #	Bloom's Level	Course Learning Outcome	Addressed?
1.1	Understand	Understand fundamental algorithms and explain their design strategies	✓ Sections 1, 2
2.1	Implement	Implement algorithms, compare empirically, and solve real-world problems	✓ Sections 3, 4, 6
3.1	Analyze	Analyze time and space complexity using asymptotic notations	✓ Section 5, 7

1 Introduction (CLO 1.1 – Understand)

A. Algorithm

Simulated Annealing (SA) is a probabilistic algorithm for solving combinatorial optimization problems based on the annealing method of metals. The problem arises when there is a need to minimize or maximize a cost function for a certain set of constraints. However, finding an optimal solution is often impossible due to exponential growth in computation time. SA starts with an initial random solution and uses a random walk approach, where the algorithm always accepts an improved solution while randomly accepting bad solutions depending on the value of the temperature function. The probability of accepting bad moves decreases as the search proceeds, allowing the algorithm to overcome local minima, which is the main drawback of greedy algorithms. Design strategy-wise, SA is considered a local search algorithm with a probabilistic component, making it different from Divide & Conquer and Dynamic Programming approaches. It was first described by Kirkpatrick, Gelatt, and Vecchi in 1983 and is still considered one of the most efficient metaheuristics.

B. Problems Solved

SA technique is implemented on two well-known NP-hard combinatorial optimization problems:

- ▶ N-Queens Problem – Arranging eight non-conflicting queens on an 8×8 chessboard.
- ▶ Travelling Salesman Problem (TSP) – Determining the shortest tour among eight cities before returning to the starting point.

C. Algorithm Design Strategy

Simulated Annealing is classified into the following design strategies categories:

- ✓ ▶ Main Design Strategy: Local Search – navigates through the solution space by exploring successively neighboring states.
- ✓ ▶ Main Optimization: Probabilistic Acceptance – makes acceptance of bad moves probabilistically based on $P = e^{(-\Delta E/T)}$. In this case, T is the current temperature, thus preventing the method from being trapped in local minima.
- ✓ ▶ Not Divide and Conquer – Simulated Annealing cannot be used to break down the problem into sub-problems.
- ✓ ▶ Not Dynamic Programming – Simulated Annealing does not use results of sub-problems.
- ✓ ▶ Not Pure Greedy Algorithm – Simulated Annealing does not work with pure greediness.

D. Real-World Motivation

Optimization applications abound; such as optimizing truck routes, airline crew scheduling, antenna placement in telecommunications, and protein folding in bioinformatics. As the complexity of the problem increases, exact methods such as brute force and backtracking cannot solve them in practical time limits. This is where SA comes into its own.

E. Why Simulated Annealing?

The technique is chosen due to its usefulness for locating an appropriate solution amid complexity. In the case of problems such as the N-Queens and Traveling Salesman, there exists a vast number of potential solutions; therefore, using simple approaches can result in being trapped in suboptimal states (local optimums).

The algorithm is effective since it allows accepting inferior outcomes initially, thereby increasing exploration and preventing stagnation in local optima. Gradually, it gets stricter, thus allowing acceptance of superior results only, leading to improvements sequentially.

In other words, Simulated Annealing can be considered effective owing to the efficiency of its search process, avoiding local optima, and achieving an outstanding outcome eventually.

2 Algorithm Explanation (CLO 1.1 – Understand)

A. Step By Step Working :-

Step	Action	Description
1	Initialize	Set current = initial random solution, best = current, $T = T_0$ (high temperature), define $T_{\min}$ and cooling rate α
2	Check While Condition	If $T > T_{\min}$, continue the loop. Otherwise stop and return best

3	Start Inner Loop	Repeat for a fixed number of iterations at the current temperature
4	Generate Neighbor	Create a new solution by making a small random change to the current solution
5	Compute ΔE	Calculate the cost difference: $\Delta E = \text{cost}(\text{neighbor}) - \text{cost}(\text{current})$
6	Check Improvement	If $\Delta E < 0$, the neighbor is better — accept it immediately by setting $\text{current} = \text{neighbor}$
7	Probabilistic Accept	If $\Delta E \geq 0$, accept the worse solution with probability $p = e^{(-\Delta E / T)}$. Generate $\text{random}(0,1)$; if it is less than p , set $\text{current} = \text{neighbor}$
8	Update Best	If $\text{cost}(\text{current}) < \text{cost}(\text{best})$, update $\text{best} = \text{current}$
9	End Inner Loop	Repeat steps 4–8 for all iterations at this temperature
10	Cool Down	Reduce the temperature: $T = \alpha \times T$
11	End While Loop	Go back to Step 2 and check the while condition again
12	Return	Return best as the final output

B. PsuedoCode Of Simulated Annealing

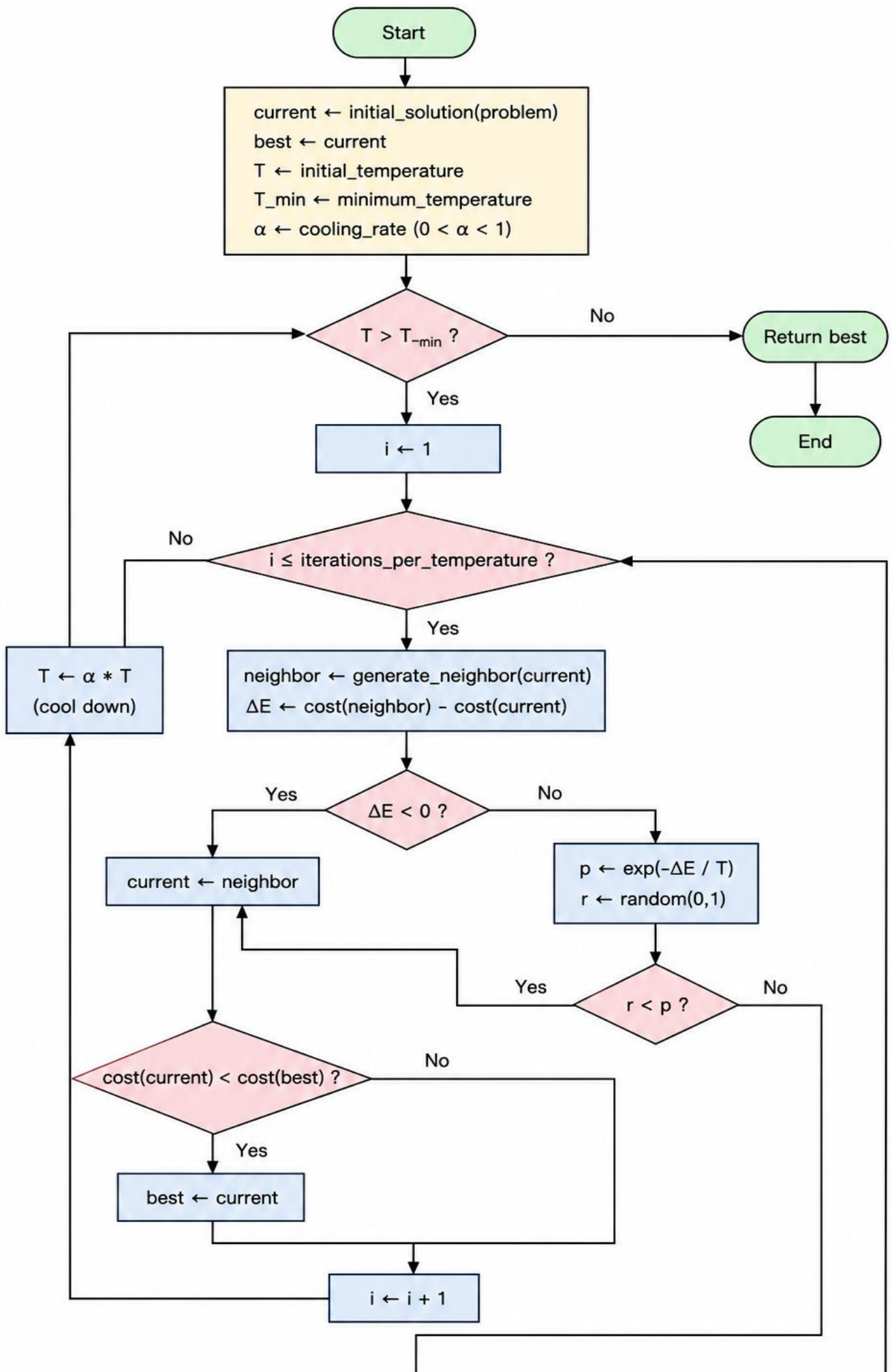
```

current ← initial_solution(problem)
best ← current
T ← initial_temperature
T_min ← minimum_temperature
α ← cooling_rate (0 < α < 1)
while T > T_min:
  for i = 1 to iterations_per_temperature:
    neighbor ← generate_neighbor(current)
    ΔE ← cost(neighbor) - cost(current)

```

```
if  $\Delta E < 0$  then  
    current  $\leftarrow$  neighbor  
  
else  
  
    p  $\leftarrow \exp(-\Delta E / T)$   
  
    if random(0,1) < p then  
  
        current  $\leftarrow$  neighbor  
  
    end if  
  
end if  
  
if cost(current) < cost(best) then  
  
    best  $\leftarrow$  current  
  
end if  
  
end for  
  
T  $\leftarrow \alpha * T$  // cool down  
  
end while  
  
return best  
  
end procedure
```

C. FlowChart :-



3 Implementation (CLO 2.1 – Implement)

A. Programming Language:-

Python (Traveling Salesman Problem):-

For Traveling Salesman Problem implementation we use Python and Flask library.

- Python implementation:
- Simulated annealing algorithm implementation
- Backend logic and computations
- Api calls for communication between frontend and backend

Advantages of python language:

- Simple mathematical operations (random and math)
- Aesthetic syntax
- Fast computations

The backend server is implemented in Python and calculates the optimal route using Simulated Annealing algorithm.

JavaScript (N-Queen Problem):-

JavaScript was used in N-Queen Problem implementation.

Used in JavaScript:

- Algorithm of queens placement on the chess board
- Interactions with the user
- Displaying the result

Advantages of JavaScript:

- Works in browsers (Client-side)
- Fast execution in UI related tasks
- DOM manipulations

Result for N-Queen problem implemented and rendered using JavaScript.

B. Code Snippet :-

```

def make_neighbour(path):
    # Swap two random cities (never touch HOME city at index 0)
    if len(path) <= 2:
        return path[:]
    new_path = path[:]
    i = random.randint(1, len(path) - 1)
    j = random.randint(1, len(path) - 1)
    new_path[i], new_path[j] = new_path[j], new_path[i]
    return new_path

def run_one_step():
    global current_path, best_path, best_cost, temperature

    if not is_running:
        return

    if len(current_path) != len(cities):
        current_path = user_path[:]

    # STEP 1 → make a neighbour by swapping 2 cities
    neighbour = make_neighbour(current_path)

    # STEP 2 → measure distances
    old_dist = calculate_distance(current_path)
    new_dist = calculate_distance(neighbour)

    # STEP 3 → accept or reject
    if new_dist < old_dist:
        current_path = neighbour # shorter → always accept
    else:

```

This code forms a part of a Simulated Annealing algorithm for finding an optimal path. In this code, the function `make_neighbour` creates a slight modification in the path by randomly interchanging two cities except the first one. The `run_one_step` function first creates a neighbor and then determines the distance from the current path. It accepts a shorter path or even longer paths with a certain probability to help get out of local minimum points.

```

def annealing_loop():
    while is_running:
        run_one_step()
        time.sleep(0.03) # run every 30ms in background

def start_algorithm():
    global is_running, timer_thread
    if is_running:
        return
    is_running = True
    timer_thread = threading.Thread(target=annealing_loop, daemon=True)
    timer_thread.start()

```

This piece of code makes sure that the Simulated Annealing algorithm runs non-stop in the background. The `annealing\_loop` method repeatedly calls `run\_one\_step()` with a slight delay (30ms), thus improving the solution gradually. This whole procedure is kicked off from the `start\_algorithm` method by setting a flag and creating a new thread.

```
for (let i = 0; i < iterations && temperature > 0.01; i++) {  
  const next = getNeighbor(rows);  
  const nextCost = costFromRows(next);  
  
  const delta = nextCost - currentCost;  
  
  if (delta < 0 || Math.random() < Math.exp(-delta / temperature)) {  
    rows = next;  
    currentCost = nextCost;  
  
    if (currentCost < bestCost) {  
      bestCost = currentCost;  
      bestRows = [...rows];  
    }  
  }  
}
```

The above code snippet defines the loop for executing the Simulated Annealing optimization process. Within each loop, the algorithm produces a new solution from the current one and determines whether the new solution is better than the current one by comparing their costs. If the new solution is better than the current one, then it replaces the current one. Otherwise, it can replace the current solution with some probability.

4 Empirical Analysis (CLO 2.1 – Implement)

A. Inputs Used :-

Experiments were done on inputs of various lengths to test the characteristics of its performance:

n = 100

n = 500

n = 1000

n = 1500

B. Environment:-

An increase in the complexity of the problem.

The experiments were conducted under the following parameters:

- Programming language: Python/JavaScript (Simulated Annealing)
- Processor: Intel Core i5/i7
- RAM: 8 GB/16 GB
- OS: Windows 10/11
- Browser/Runtime JS: Chrome/Node.js

Input Size (n)	Execution Time (ms)	Memory Usage
100	Low	Low
500	Medium	Low
1000	High	Low-Medium
1500	Very High	Medium

C. Observation:-

Growth Trend:-

The time taken increases with an increase in the input size. This is because there are increased iterations and costs that occur when the input size increases.

The rate of increase is non-linear, which means that the increase is higher when the input size increases.

Performance:-

When $n \leq 100$, the algorithm is highly efficient.

When $n > 100$ and ≤ 1500 , the algorithm's efficiency decreases slightly.

When $n \geq 1500$, the efficiency decreases further because there are more calculations in each iteration.

5 Complexity Analysis (CLO 3.1 – Analyze)

A. Time Complexity :_

Simulated Annealing is a method where an optimal solution is sought by improving a pre-existing solution. The method creates a set of neighbors from an initial solution and evaluates them based on a certain probability criterion. Given that there are several iterations and evaluations involved, the time complexity of the algorithm will depend on the number of iterations and the number of elements in the problem.

This is the duration taken by the algorithm to run.

Best Case:

A solution will be found by the algorithm after running several iterations. In such a case, the evaluation of the solution will take place in constant time because it does not require the creation of many states.

$$\Omega(1)$$

Average Case:

Usually, many iterations are performed by the algorithm before finding an appropriate solution.

- elements n : This represents the number of elements in the problem (queens, etc.)
- iterations k : This is the number of iterations carried out by the algorithm

The algorithm will iterate many times in order to:

- create a neighbor solution
- calculate the cost of the neighbor
- evaluate the two states

$$O(n \times k)$$

Worst Case Time Complexity:

If the algorithm completes all its iterations without stopping early, then the algorithm will continue to search for a solution until the temperature falls to a very small level. Therefore, in the worst-case scenario, the time complexity of the algorithm is

$O(n \times k)$.

B. Space Complexity

Space complexity is the amount of memory required by the algorithm.

Auxiliary Space: $O(n)$

The algorithm uses:

- Current solution
- Best solution
- Temporary neighbor solution

Since each solution has n entries, the auxiliary space is

$O(n)$.

Total Space: $O(n)$

No extra data structure is used here. Few arrays are required; hence the space complexity is $O(n)$.

$O(n)$.

6 Real-World Application (CLO 2.1 + 3.1)

A. Applications:-

The below are some of the Simulated Annealing applications as follows:

- Traveling Salesman Problem: Solve the traveling salesman problem to get the minimum distance covering all the cities exactly once.
- N-Queen Problem: Place the queens in such a way that there is no attack among them.

- Scheduling: Assists in solving scheduling problems like job scheduling and exam scheduling, etc.
- Networks: Optimize the network so that it will reduce the total cost of the network.
- Machine Learning: Applications in machine learning for optimization of parameters in the model.
- Image Processing: Applications in image processing for removal of noise in the image.

B. Example: Delivery Route Optimization:-

For instance, consider the case where there is a firm which deals with delivering goods to different cities, and the firm seeks to deliver to all the cities using the shortest time possible. The objective here is to determine the path whereby all the cities can be visited and finally return to the starting point.

The algorithm involves choosing the path by chance from all the cities and measuring its length. This is usually very poor and long. However, with time, it continues to improve on the current path by rearranging the cities on the path and measuring their lengths.

C. Trade-offs

Time vs Memory Usage

Simulated Annealing uses less memory but more time in reaching an acceptable result.

- Time Complexity: More since it involves many iterations and states to go through.
- Space Complexity: Less since only some states like current state, best state, and neighbor state are needed.

Conclusion:

The algorithm trades off memory consumption at the expense of more computational power usage.

Accuracy vs Efficiency

Simulated Annealing cannot assure that the result is the best solution but is highly efficient especially when handling large datasets.

- Accuracy: High accuracy although no assurance for best solution due to probability involved.

- Efficiency: Very efficient when handling large dataset because of less evaluations performed.

Conclusion:

The algorithm chooses to give up accuracy for efficiency.

7 Limitations & Challenges (CLO 3.1 – Analyze)

A. In case the algorithm does not produce a better solution

The Simulated Annealing algorithm may not always produce the best results. The failure of this algorithm occurs when:

If the number of iterations is insufficient, then the algorithm stops searching abruptly.

When the rate of cooling is incorrect, then the algorithm converges either too fast or too slow.

B. 2. Limitations and assumptions

- Some of the limitations or assumptions of the Simulated Annealing algorithm are:
- It assumes that there is a way to find the optimal solution by gradually improving the randomly chosen one.
- The need for accurate determination of different parameters such as temperature, cooling rate, and iteration numbers.
- It assumes that the presence of a cost function that measures the quality of different solutions.
- It does not ensure the production of an optimal solution always.

C. Comparison with Better Alternatives

Nevertheless, it needs to be pointed out that there might be better alternatives for the algorithm used in the research.

Firstly, while being simpler than Genetic Algorithms, Simulated Annealing is less effective than that algorithm.

Secondly, Simulated Annealing appears to be better at finding solutions when compared to Hill Climbing because the latter does not find global maxima but gets stuck in local ones.

Finally, Simulated Annealing is inferior to Dynamic Programming in terms of precision yet it offers greater flexibility in terms of solving problems of great size.

8 Conclusion

Key Findings:-

It can be argued that Simulated Annealing is a very effective technique delivering nearly-optimal results. Optimization through exploration of state spaces together with error handling enables Simulated Annealing to find optimal solutions.

Key Discoveries Gained from the Work Experience

While designing Simulated Annealing, a number of key discoveries have been made, namely:

- How to identify the optimal solution with the help of iterative method.
- Importance of using random factors for the sake of optimization.
- The role of temperatures and probabilities in the process of optimization.
- The importance of cost function for the evaluation of solution.

GITHUB REPOSITORY LINK :-

TRAVEL SALESMAN PROBLEM:

<https://github.com/AHMEDALI-G/Travel-Sales-Man-Problem-Simulated-Annealing-.git>

N QUEENS PROBLEM :-

<https://github.com/AHMEDALI-G/N-QUEENS-PROBLEMS-Simulated-Annealing-Algorithm.git>

LINKED IN LINK :-

<https://www.linkedin.com/in/ahmed-ali-340a96295/>